

Power and Test Design

Math 742

2026-03-06

Introduction

Power matters, there is no free lunch.

So far we have focused on simple hypotheses and mechanics under the null. Now, we shift to composite hypotheses (where H_0 or H_1 spans a range of parameters) and emphasize power, the probability of correctly rejecting H_0 when it's false. In data science, power analysis is crucial for designing experiments like A/B tests or setting drift thresholds, ensuring you can detect meaningful effects without wasting resources.

Consider a radar system used to detect enemy aircraft.

H_0 : No enemy aircraft present

H_1 : An enemy aircraft is in your air space at some distance, speed, etc.

Significance level (α) equals the probability that a radar falsely signals an aircraft when there actually isn't an aircraft (i.e. false alarm).

Power is the probability that the radar signals an aircraft when one is actually present.

Key insight

- A plane that is large and close (strong signal) is easy to detect $\rightarrow$ high power
- A plane that is small or far away (weak signal) is hard to detect $\rightarrow$ lower power

Most likely there is no way to tune the radar so that it can simultaneously detect slow planes, fast planes and stealth planes. In other words, it is unlikely that a uniformly most power test exists.

Power depends on which alternative is true, not just that the alternative is true.

Composite Hypotheses

The parameter space consists of $\Theta = \Theta_0 \cup \Theta_1$, where $\Theta_0 \cap \Theta_1 = \emptyset$

- $H_0 : \Theta \in \Theta_0$
- $H_1 : \Theta \in \Theta_1$

Power is either point-wise or uniform.

- Point-wise: Power is evaluated at a specific value of Θ in Θ_1 .
- Uniform: Power holds for all Θ in Θ_1 simultaneously (stronger but rarer).

Most tests are point-wise optimal (best at specific alternatives) but not uniformly most powerful (UMP) across all alternatives, no “free lunch” test dominates everywhere.

Example: Testing a shift in mean for a normal distribution.

$$H_0 : \mu \leq 0 \text{ vs. } H_1 : \mu > 0 \text{ (composite alternative)}$$

The one-sided t-test is UMP here (monotone LR property), but for two-sided $H_1 : \mu \neq 0$, no UMP test exists.

Power Function

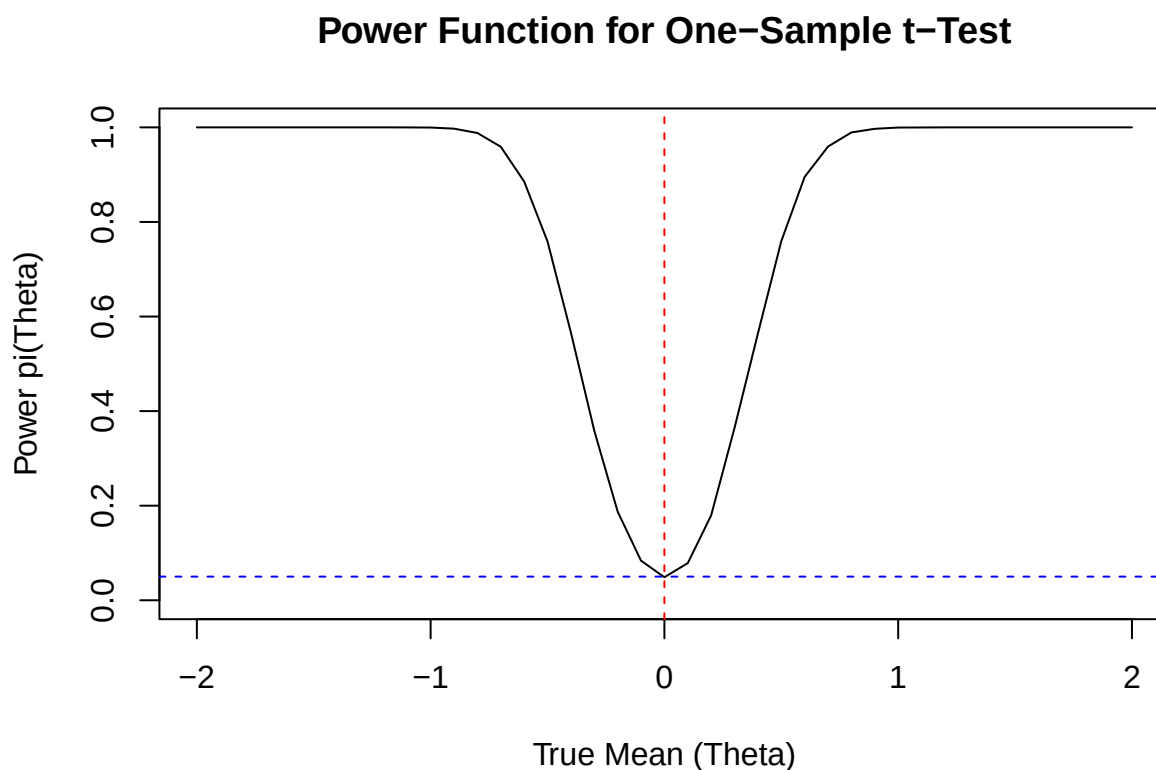
$$\pi(\Theta) = \mathbb{P}_{\Theta}(\text{reject } H_0)$$

- For $\Theta \in \Theta_0 : \pi(\Theta) = \mathbb{P}$ (Type I error).
- For $\Theta \in \Theta_1 : \pi(\Theta) = 1 - \beta(\Theta) = \mathbb{P}$ power at Θ , we want this to be high.

Power Curve

Power curves plot the power versus different values of Θ . The ideal power curve will be low in H_0 in order to control false positives, will rise sharply in H_1 in order to detect.

Example of a power curve:



Notice that near zero there is low power to detect a difference from 0. Further from zero power increases.

No test maximizes power everywhere while controlling size—trade-offs are inevitable.

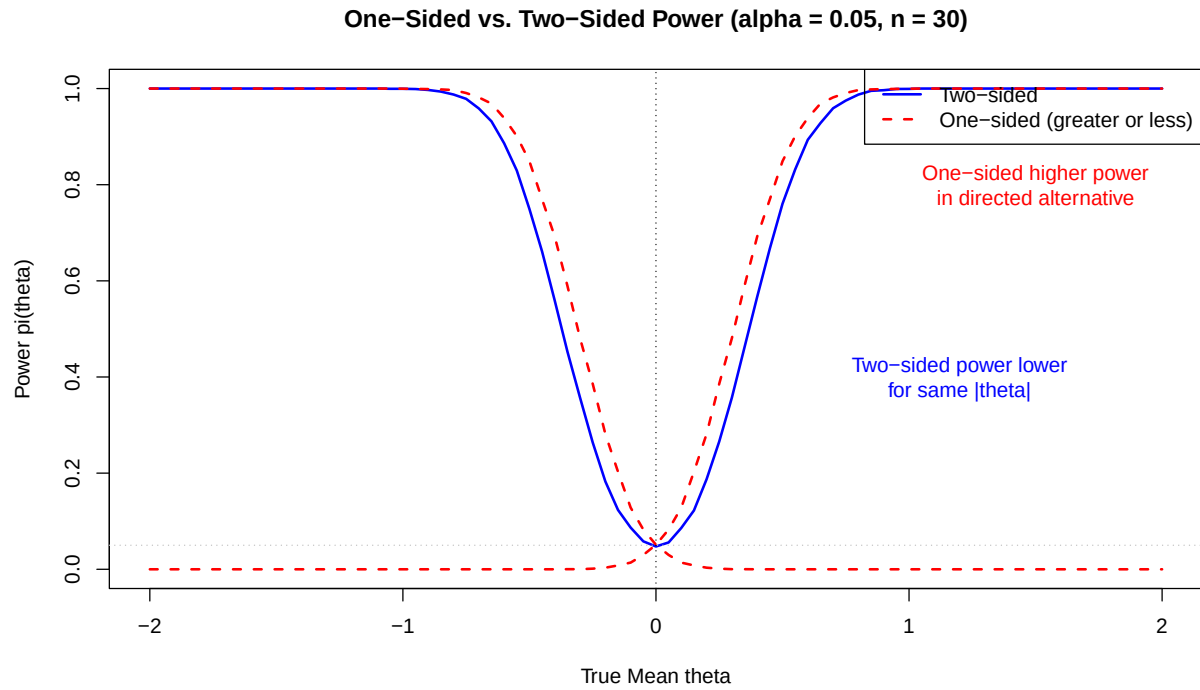


Figure 1: Power Functions: One-Sided vs. Two-Sided t-Test ($n=30$, $\sigma=1$, $\alpha=0.05$)

Power vs. Size (α)

- Smaller α (e.g., 0.01 vs. 0.05) reduces false positives but lowers power (higher β).
- In DS: For high-stakes (e.g., medical ML), low α ; for exploratory (e.g., feature screening), higher α ok.

Recall Day 1: Testing as risk minimization. Power ($1 - \beta$) minimizes expected loss from Type II errors, balanced against α (Type I loss).

In asymmetric DS settings (e.g., false positive in fraud detection costs more), adjust α/β via costs λ_1/λ_2 .

What else affects power?

Change	Effect on Power
Increase α	Power increases
Increase n	Power increases
Decrease σ	Power increases
Increase d	Power increases
2-tailed instead of 1-tailed	Power decreases

For illustration purposes, I will use a one-sample t-test.

$$H_0 : \mu = 10, H_1 : \mu = 11$$

```
set.seed(123)
# Given values
alpha = 0.05
n = 30
s = 3.5
mu_0 = 10
```

```

mu_true = 11

# Calculate the critical t value for a one-tailed test at alpha = 0.05
df = n - 1
t_crit = qt(alpha,df,lower.tail=FALSE)

# Standardize: (mu_true = 11)
standard_t = (mu_true - mu_0) / (s / sqrt(n))

# Calculate beta
beta = pt(t_crit, df, standard_t)
# Calculate the power using the t distribution
power = 1 - beta

# Display the power
power

```

```
## [1] 0.4536073
```

Now, increase α and see what happens to power.

```

set.seed(123)
# Given values
alpha = 0.1 # increase to 0.1
n = 30
s = 3.5
mu_0 = 10
mu_true = 11

# Calculate the critical t value for a one-tailed test at alpha = 0.05
df = n - 1
t_crit = qt(alpha,df,lower.tail=FALSE)

# Standardize: (mu_true = 11)
standard_t = (mu_true - mu_0) / (s / sqrt(n))

# Calculate beta
beta = pt(t_crit, df, standard_t)
# Calculate the power using the t distribution
power = 1 - beta

# Display the power
power

```

```
## [1] 0.6029677
```

Power increases.

Now decrease the standard deviation.

```

set.seed(123)
# Given values
alpha = 0.05
n = 30
s = 1.5 # decrease to 1.5
mu_0 = 10

```

```

mu_true = 11

# Calculate the critical t value for a one-tailed test at alpha = 0.05
df = n - 1
t_crit = qt(alpha,df,lower.tail=FALSE)

# Standardize: (mu_true = 11)
standard_t = (mu_true - mu_0) / (s / sqrt(n))

# Calculate beta
beta = pt(t_crit, df, standard_t)
# Calculate the power using the t distribution
power = 1 - beta

# Display the power
power

```

```
## [1] 0.9725412
```

Double the sample size. See what happens to power.

```

set.seed(123)
# Given values
alpha = 0.05
n = 60 # double sample size
s = 3.5
mu_0 = 10
mu_true = 11

# Calculate the critical t value for a one-tailed test at alpha = 0.05
df = n - 1
t_crit = qt(alpha,df,lower.tail=FALSE)

# Standardize: (mu_true = 11)
standard_t = (mu_true - mu_0) / (s / sqrt(n))

# Calculate beta
beta = pt(t_crit, df, standard_t)
# Calculate the power using the t distribution
power = 1 - beta

# Display the power
power

```

```
## [1] 0.7063616
```

Increase effect size.

```

set.seed(123)
# Given values
alpha = 0.05
n = 30
s = 3.5
mu_0 = 10
mu_true = 12

```

```

# Calculate the critical t value for a one-tailed test at alpha = 0.05
df = n - 1
t_crit = qt(alpha,df,lower.tail=FALSE)

# Standardize: (mu_true = 11)
standard_t = (mu_true - mu_0) / (s / sqrt(n))

# Calculate beta
beta = pt(t_crit, df, standard_t)
# Calculate the power using the t distribution
power = 1 - beta

# Display the power
power

## [1] 0.9208311

```

Built-in power functions R and Python

In R there is a package called “pwr”. There are various functions in this package that will help with power functions depending on the test (e.g. one/two sample t-test, ANOVA, proportion tests). Basically, the functions work mostly the same, you provide 3 of the following 4 items. The fourth item is solved numerically.

1. d, Effect size $d = (\mu - \mu_0) / \sigma$
2. n, Number of observations
3. sig.level, Significance level (Type I error probability)
4. power, Power of test (1 minus Type II error probability)

Note: you also need to specify which alternative you are testing.

alternative, a character string specifying the alternative hypothesis, must be one of “two.sided” (default), “greater” or “less”

```

library(pwr)
pwr.t.test(
  n = 30,
  d = (11 - 10) / 3.5, # effect size
  sig.level = 0.05,
  type = "one.sample",
  alternative = "greater"
)

```

```

##
##      One-sample t test power calculation
##
##              n = 30
##              d = 0.2857143
##      sig.level = 0.05
##      power = 0.4536073
##      alternative = greater

```

Python equivalent:

```

from statsmodels.stats.power import TTestPower

```

```

effect_size = (11 - 10) / 3.5

power = TTestPower().power(
    effect_size=effect_size,
    nobs=30,
    alpha=0.05,
    alternative='larger'
)

print(power)

```

```
## 0.4536073486413207
```

Applications

1. A/B testing. In the context of A/B testing (or marketing/analytics in general), “lift” refers to the relative or absolute increase in a metric (like conversion rate) that you hope to detect due to the new version (B) compared to the control (A).

Design an A/B test with the following specifications:

- Baseline conversion: $p_0 = 0.10$
- Desired lift: $\Delta = 0.03 \rightarrow p_2 = 0.13$
- Significance Level: $\alpha = 0.05$ (two-sided)
- Power: $1 - \beta = 0.90$

```

library(pwr)

# Parameters
p1 <- 0.10      # baseline
p2 <- 0.13      # variant (lift)
sig.level <- 0.05
power <- 0.90

# Compute effect size for proportions
h <- ES.h(p1, p2) # Cohen's h

# Sample size per group
pwr.2p.test(h = h, sig.level = sig.level, power = power, alternative = "two.sided")

##
##      Difference of proportion power calculation for binomial distribution (arcsine transformation)
##
##              h = 0.09422486
##              n = 2366.983
##      sig.level = 0.05
##      power = 0.9
##      alternative = two.sided
##
## NOTE: same sample sizes

```

So, we need 2367 samples per group.

2. Nested model comparison

Suppose you are predicting sales using:

- Model 0 (baseline): $\text{sales} \sim 1$ (intercept only model)
- Model 1 (full): $\text{sales} \sim \text{ad_spent} + \text{social_media}$

We want to test if adding `ad_spend` and `social_media` significantly improves the model.

$$H_0 : \beta_1 = \beta_2 = 0$$

$$H_1 : \text{at least one } \beta \text{ is non-zero}$$

Perform the likelihood ratio test. For nested models, the test statistic is:

$$\Lambda = -2(\log L(\text{reduced model}) - \log L(\text{full model}))$$

Under the null hypothesis, the test statistic has a χ^2 distribution with (parameters full model) - (parameters reduced model). Full model has intercept + two slopes = 3 parameters, the reduced model has the intercept. Difference = 3 - 1 = 2.

Simulate:

```
set.seed(123)
n <- 50
ad_spend <- rnorm(n, 50, 10)
social_media <- rnorm(n, 30, 5)
sales <- 100 + 2*ad_spend + 1.5*social_media + rnorm(n, 0, 10)

# Fit models
model0 <- lm(sales ~ 1)
model1 <- lm(sales ~ ad_spend + social_media)

# LRT
lrt_stat <- 2 * (logLik(model1) - logLik(model0))
p_value <- pchisq(lrt_stat, df=2, lower.tail = FALSE)
lrt_stat; p_value
```

```
## 'log Lik.' 79.45158 (df=4)
```

```
## 'log Lik.' 5.588669e-18 (df=4)
```

Note: the df in the printout is not the degrees of freedom for χ^2 . This is the number of parameters estimated (intercept, `ad_spend`, `social_media`, residual variance).

Here the log-likelihood is quite larger so we reject H_0 , p-value is quite small.

Suppose the true effect is small: $\beta_1 = 0.5$ and $\beta_2 = 0.3$. If we simulate many datasets we can see how often the LRT detects improvement. Essentially, we are estimating power.

```
n_sim <- 1000
power_count <- 0

for (i in 1:n_sim) {
  sales_sim <- 100 + 0.5*ad_spend + 0.3*social_media + rnorm(n, 0, 10)
  m0 <- lm(sales_sim ~ 1)
  m1 <- lm(sales_sim ~ ad_spend + social_media)
  lrt <- 2*(logLik(m1) - logLik(m0))
  if (pchisq(lrt, df=2, lower.tail=FALSE) < 0.05) power_count <- power_count + 1
}
```



```
power_estimate <- power_count / n_sim
power_estimate
```

```
## [1] 0.859
```

- power_estimate tells you probability of detecting the improvement with this sample size and effect.
- Small effect $\rightarrow$ lower power $\rightarrow$ may need larger n to detect reliably. Here, power is fairly high so n is probably adequate.

3. Drift detection thresholds

H_0 : No drift in data distribution, or small drift less than η

H_1 : Drift $> \eta$

We will use something called a KS-test which uses as its test statistic the largest distance between the 2 densities.

```
set.seed(123)

# --- Step 0: generate baseline data ---
n <- 100
x_ref <- rnorm(n, mean = 0, sd = 1) # reference distribution

# --- Step 1: simulate a "new" batch under null ---
x_new_null <- rnorm(n, mean = 0, sd = 1) # no drift

ks.test(x_ref, x_new_null)
```

```
##
## Asymptotic two-sample Kolmogorov-Smirnov test
##
## data: x_ref and x_new_null
## D = 0.13, p-value = 0.3667
## alternative hypothesis: two-sided
```

Set threshold for desired power:

```
# --- Step 2: simulate under alternative ---
n_sim <- 5000
shift <- 0.5
eta <- 0.05
detections <- numeric(n_sim)

for(i in 1:n_sim){
  x_new_alt <- rnorm(n, mean = shift, sd = 1)
  detections[i] <- ks.test(x_ref, x_new_alt)$p.value < eta
}

power_estimate <- mean(detections)
power_estimate
```

```
## [1] 0.754
```

Failures of optimality

For composite hypotheses, UMP tests rarely exist. Different alternatives favor different test.

Example:

Let $X_1, X_2, \dots, X_n$ be iid $N(\mu, \sigma^2)$ where $\sigma^2 < \infty$ is known. We want to perform to test the following composite alternative hypothesis.

$$H_0 : \mu = \mu_0 \text{ vs. } H_1 : \mu \neq \mu_0$$

- For the 2-sided test we would reject H_0 when $|\bar{X} - \mu_0|$ is too large.
- Statistic: $Z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}}$
- Reject if $|Z| > z_{1-\alpha/2}$

There is no UMP for this test.

- The Neyman-Pearson lemma guarantees a UMP test only for simple vs. simple or one-sided alternatives.
- For two-sided alternatives, the optimal rejection region would depend on the unknown true direction of the effect.
- So there is no uniformly most powerful (UMP) test for all possible two-sided shifts.

Hypothesis testing is not just about rejecting H_0 ; it is about designing tests that have enough power to detect meaningful effects.

Hypothesis Testing: 5-Day Course Summary

Day	Focus	Core Takeaways
1	Motivation/Framework	HT as error-controlled decisions in DS (e.g., model deployment, A/B). Errors (Type I/II), significance α , risk minimization. No rituals—focus on reproducibility and guardrails.
2	Basics of a HT	Test stats summarize evidence (invariance for known distributions under H_0). Rejection regions control size; p-values measure incompatibility (not $P(H_0 \text{ true})$)—ASA guidelines). Classical examples: Z/t-tests for means. CI duality.
3/4	Optimality/LRs	Likelihood ratios unify optimal tests (NP lemma) and asymptotic tests (Wilks theorem): $-2 \log \Lambda \sim \chi_q^2$ asymptotically). Intuitive proof via Taylor + normality. Equivalence to Wald/score. Optimal = max power at fixed α , but caveats (misspecification).
5	Composite Hypothesis/Power	Power = $\pi(\Theta) = \mathbb{P}(\text{reject } H_0 \text{ when false})$

Key Concept Table

Null Hypothesis	Alternate Hypothesis	Significance Level	p-value	Critical Region
The default assumption or claim that there is no significant difference or effect.	The assertion that contradicts the null hypothesis suggesting there is a significant difference or effect.	The threshold that is used to determine whether or not to reject the null hypothesis	The probability of observing the data or more extreme results given that the null hypothesis is true.	The range of values that, if the test statistic falls within it, leads to the rejection of the the null hypothesis.

Python Code

```
import numpy as np
from scipy.stats import t, nct

# -----
# Base case
# H0:  $\mu = 10$ , H1:  $\mu = 11$ 
# -----
alpha = 0.05
n = 30
s = 3.5
mu_0 = 10
mu_true = 11

df = n - 1

# Critical value (one-sided)
t_crit = t.ppf(1 - alpha, df)

# Noncentrality parameter
delta = (mu_true - mu_0) / (s / np.sqrt(n))

# Beta and power
beta = nct.cdf(t_crit, df, delta)
power = 1 - beta

print("Power (baseline):", power)

## Power (baseline): 0.45360734864132124

Increase  $\alpha$ 
alpha = 0.10

t_crit = t.ppf(1 - alpha, df)
beta = nct.cdf(t_crit, df, delta)
power = 1 - beta

print("Power (alpha = 0.10):", power)

## Power (alpha = 0.10): 0.6029676666793389

Decrease standard deviation
```

```

alpha = 0.05
s = 1.5

delta = (mu_true - mu_0) / (s / np.sqrt(n))
t_crit = t.ppf(1 - alpha, df)

beta = nct.cdf(t_crit, df, delta)
power = 1 - beta

print("Power (s = 1.5):", power)

```

```
## Power (s = 1.5): 0.9725411623767679
```

Double sample size

```

n = 60
s = 3.5
df = n - 1

delta = (mu_true - mu_0) / (s / np.sqrt(n))
t_crit = t.ppf(1 - alpha, df)

beta = nct.cdf(t_crit, df, delta)
power = 1 - beta

print("Power (n = 60):", power)

```

```
## Power (n = 60): 0.7063616019705591
```

Increase effect size

```

n = 30
s = 3.5
mu_true = 12
df = n - 1

delta = (mu_true - mu_0) / (s / np.sqrt(n))
t_crit = t.ppf(1 - alpha, df)

beta = nct.cdf(t_crit, df, delta)
power = 1 - beta

print("Power (mu_true = 12):", power)

```

```
## Power (mu_true = 12): 0.9208311071817056
```